ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 20

Namespaces, Composer y autocarga

- El problema: require_once por todas partes y nombres que chocan
- Namespaces: el apellido de las clases
- use: llamar a las clases por su nombre de pila
- La autocarga, hecha a mano
- PSR-4: la regla que lo ordena todo
- Composer: el gestor de PHP
- Instalar librerías de otros: Carbon
- composer.json, composer.lock y vendor
- Los errores típicos de hoy



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

En las dos últimas clases os he ido prometiendo lo mismo: «esto de cargar las clases a mano con `require_once`, y además en orden, lo arreglaremos». Ha llegado el día. 🎉

Hoy aprendemos tres cosas que van juntas y que usa **cualquier proyecto PHP profesional**, Laravel incluido:

- **Namespaces**, para que dos clases puedan llamarse igual sin chocar.
- **Autocarga**, para que PHP cargue cada clase solo, en el momento en que la necesita.
- **Composer**, para organizar el proyecto e instalar librerías de otros con un solo comando.

1 🤔 El problema: require_once por todas partes y nombres que chocan



2 📘 Namespaces: el apellido de las clases



3 👍 use: llamar a las clases por su nombre de pila



4 🔍 La autocarga, hecha a mano



5 👌 PSR-4: la regla que lo ordena todo



6 🚀 Composer: el gestor de PHP



7 😮 Instalar librerías de otros: Carbon



8 ⚠️ composer.json, composer.lock y vendor



9 🐞 Los errores típicos de hoy



10 🚀 Ejercicios



11 👋 Lo que tenéis que llevaros de hoy





CLASE 20 · PUNTO 1

🤔 El problema: require_once por todas partes y nombres que chocan

Recordad cómo empezaba un archivo en la clase 18:

ejemplo.php — php-paso-a-paso/clase20 — Visual Studio Code

ejemplo.php

```php
<?php
require_once __DIR__ . "/clases/Usuario.php";    // el padre, primero
require_once __DIR__ . "/clases/Alumno.php";
require_once __DIR__ . "/clases/Profesor.php";
```

Con tres clases, vale. Con cincuenta, cada archivo empezaría con cincuenta líneas de `require_once`, y además **en el orden correcto**. Una pesadilla.

Y hay un segundo problema: los **nombres**. Imaginad que vuestra academia tiene una clase `Usuario` y descargáis una librería de foros que **también** tiene una clase `Usuario`. PHP no puede tener dos clases con el mismo nombre: *Cannot declare class Usuario, because the name is already in use*.

Hoy resolvemos las dos cosas.



CLASE 20 · PUNTO 2

📘 Namespaces: el apellido de las clases

En una clase puede haber dos alumnas llamadas Marta, y no pasa nada: una es Marta García y la otra, Marta López. El **namespace** (espacio de nombres) es el **apellido** de una clase [1]:

01-nombres-que-chocan.php — php-paso-a-paso/clase20 — Visual Studio Code

01-nombres-que-chocan.php

```php
<?php
declare(strict_types=1);

namespace App\Modelos;

class Usuario
{
    public function __construct(public string $nombre)
    {
    }
}
```

Ahora el nombre completo de la clase ya no es `Usuario`, sino **`App\Modelos\Usuario`**. Y el `Usuario` de la librería de foros puede ser `Foro\Entidades\Usuario`. Dos clases distintas, sin choque.

Regla	Ejemplo
`namespace` va al principio, justo después de `declare(strict_types=1);`	`namespace App\Modelos;`
Las partes se separan con la barra **invertida** `\`	`App\Modelos`, `App\Servicios`
Cada parte empieza por mayúscula, como las clases	`App\Modelos`, no `app\modelos`
Un namespace por archivo	
`Clase::class` devuelve el nombre completo	`Usuario::class` → `"App\Modelos\Usuario"`

📄 Ejemplo: `01-nombres-que-chocan.php` (solo para el ejemplo mete los dos namespaces en un archivo)



CLASE 20 · PUNTO 3

👍 use: llamar a las clases por su nombre de pila

Escribir `new \App\Modelos\Usuario(...)` cada vez sería agotador. Con **`use`**, al principio del archivo, le decís a PHP: «cuando escriba `Usuario`, me refiero a este» [2].

ejemplo.php — php-paso-a-paso/clase20 — Visual Studio Code

ejemplo.php

```php
<?php
declare(strict_types=1);

use App\Modelos\Usuario;
use Foro\Entidades\Usuario as UsuarioForo;   // un alias para no chocar

$marta = new Usuario("Marta García");
$martaEnElForo = new UsuarioForo("marta_dev");
```

Es como en una reunión de amigos: una vez que habéis dicho «Marta es Marta García», ya la llamáis Marta. Y si hay dos Martas, a una la llamáis por un mote (`as`).

⚠️ La trampa de las clases de PHP

Dentro de un archivo con namespace, PHP busca **todas** las clases en ese namespace. Si en `App\Modelos` escribís `new DateTime()` , PHP buscará `App\Modelos\DateTime` … y no lo encontrará. Las clases que vienen con PHP (`DateTime` , `PDO` , `Exception` …) viven en el espacio **global**, y hay dos formas de llegar a ellas:

ejemplo.php — php-paso-a-paso/clase20 — Visual Studio Code

ejemplo.php

```php
<?php
namespace App\Modelos;

use DateTimeImmutable;                  // 1. importándola con use (la que yo prefiero)

$alta = new DateTimeImmutable();
$conexion = new \PDO($dsn, $usuario, $clave);   // 2. o con la barra delante: \PDO
```



🎯 Si una clase está en el **mismo** namespace que el archivo, no hace falta `use` . En las soluciones, `Alumno` y `Usuario` están los dos en `App\Usuarios` , así que `class Alumno extends Usuario` funciona sin importar nada.



CLASE 20 · PUNTO 4

🔍 La autocarga, hecha a mano

Vamos con el otro problema, el de los cincuenta `require_once` . PHP tiene una función, `spl_autoload_register()` , para decirle: «cuando te encuentres una clase que no conoces, llama a esta función antes de dar error» [3]. Y esa función es la que busca y carga el archivo.

— php-paso-a-paso/clase20 — Visual Studio Code

```php
<?php
declare(strict_types=1);

use App\Modelos\Producto;
use App\Servicios\Carrito;

spl_autoload_register(function (string $clase): void {
    // "App\Servicios\Carrito"  →  "src/Servicios/Carrito.php"
    $relativa = substr($clase, strlen("App\\"));
    $archivo = __DIR__ . "/src/" . str_replace("\\", "/", $relativa) . ".php";
    if (file_exists($archivo)) {
        require $archivo;
    }
});

$carrito = new Carrito();                    // PHP no conoce Carrito → llama a la función
$carrito->anadir(new Producto("Camiseta", 15));   // y lo mismo con Producto
```

Ni un solo `require` de clases. Y el orden ya no importa: si `Alumno` hereda de `Usuario`, en cuanto PHP lee `extends Usuario` pide `Usuario` a la autocarga.

📄 Ejemplo: `autocarga-manual/`. Su función de autocarga escribe un mensaje cada vez que carga una clase, para que veáis cuándo lo hace.



💡 No vais a escribir esta función en vuestros proyectos: lo hará Composer por vosotros. Pero quiero que sepáis que **no hay magia**: la autocarga solo traduce el nombre de la clase a una ruta y hace un `require`.



CLASE 20 · PUNTO 5

👌 PSR-4: la regla que lo ordena todo

Para que la autocarga funcione, el nombre de la clase y la ruta del archivo tienen que **corresponderse**. Esa correspondencia está estandarizada: se llama **PSR-4** y la sigue todo el mundo, Laravel incluido [4].

La regla es: «el namespace `App\` corresponde a la carpeta `src/`, y a partir de ahí, **cada parte del namespace es una carpeta** y el nombre de la clase es el nombre del archivo».

Nombre completo de la clase	Archivo
`App\Modelos\Producto`	`src/Modelos/Producto.php`
`App\Modelos\Pedido`	`src/Modelos/Pedido.php`
`App\Servicios\Carrito`	`src/Servicios/Carrito.php`
`App\Servicios\Pagos\Tarjeta`	`src/Servicios/Pagos/Tarjeta.php`

Esquema

```
proyecto-composer/
├── composer.json
├── index.php
├── src/                        ← App\
│   ├── Modelos/                ← App\Modelos
│   │   ├── Pedido.php          ← App\Modelos\Pedido
│   │   └── Producto.php        ← App\Modelos\Producto
│   └── Servicios/              ← App\Servicios
│       └── Formato.php         ← App\Servicios\Formato
└── vendor/                     ← lo crea Composer (no se toca)
```



⚠️ **Mayúsculas y minúsculas importan.** En Windows, `src/modelos/producto.php` funcionará aunque el namespace sea `App\Modelos\Producto`, pero en un servidor Linux, **no**. Acostumbraos desde hoy a que coincidan exactamente.



CLASE 20 · PUNTO 6

🚀 Composer: el gestor de PHP

Composer es la herramienta oficial de PHP para dos cosas: configurar la **autocarga** de vuestro proyecto e **instalar librerías** de otros programadores [5]. Laragon ya lo trae instalado: pulsad el botón **Terminal** y escribid:

terminal — php-paso-a-paso/clase20 — Visual Studio Code

terminal

```
composer -V
```

Paso 1 · El archivo composer.json

En la raíz del proyecto, un archivo `composer.json` describe el proyecto. Para la autocarga, basta con esto:

datos.json — php-paso-a-paso/clase20 — Visual Studio Code

datos.json

```
{
    "name": "alpe/tienda",
    "autoload": {
        "psr-4": {
            "App\\": "src/"
        }
    }
}
```

Leedlo como la regla del punto anterior: «el namespace `App\` está en la carpeta `src/`». La barra va doble (`App\\`) porque en JSON la barra invertida se escribe así.

Paso 2 · Generar la autocarga

Desde la terminal, en la carpeta del proyecto:

terminal — php-paso-a-paso/clase20 — Visual Studio Code

terminal

```
composer dump-autoload
```

Composer crea la carpeta `vendor/` con un archivo, `vendor/autoload.php`, que hace exactamente lo que hacía nuestra función del punto 4.

Paso 3 · Una sola línea en vuestro programa

— php-paso-a-paso/clase20 — Visual Studio Code

```php
<?php
declare(strict_types=1);

require_once __DIR__ . "/vendor/autoload.php";

use App\Modelos\Pedido;
use App\Modelos\Producto;

$pedido = new Pedido(1001);
$pedido->anadir(new Producto("Camiseta", 15), 2);
```

Un único `require_once`, tengáis cinco clases o quinientas.

📄 Ejemplo: `proyecto-composer/`



💡 Si creáis una clase nueva siguiendo PSR-4, normalmente no hay que hacer nada más. `composer dump-autoload` solo hace falta cuando cambiáis la sección `autoload` del `composer.json`.

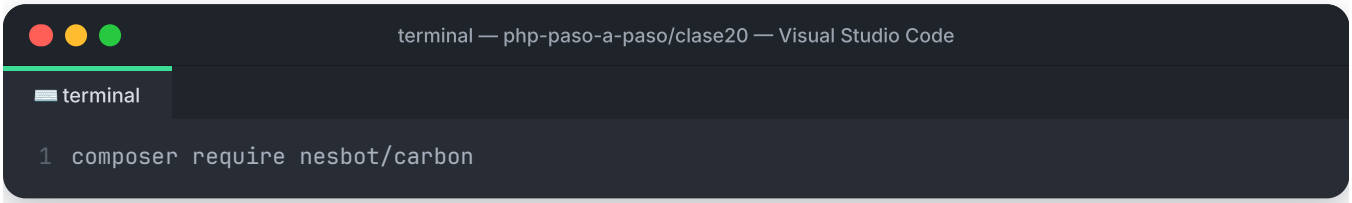


CLASE 20 · PUNTO 7

😮 Instalar librerías de otros: Carbon

La otra mitad de Composer es la que lo hizo famoso: **instalar librerías** (paquetes) que han escrito otros. En packagist.org hay cientos de miles. ¿Para qué reinventar la rueda?

Vamos a instalar **Carbon**, la librería de fechas más usada de PHP (la usa Laravel por dentro):



Composer descarga Carbon (y lo que Carbon necesite) en `vendor/`, lo apunta en el `composer.json` y lo añade a la autocarga. Ya podéis usarlo con un `use`:

ejemplo.php — php-paso-a-paso/clase20 — Visual Studio Code

ejemplo.php

```php
<?php
namespace App\Modelos;

use Carbon\Carbon;

class Pedido
{
    private Carbon $fecha;

    public function __construct(public readonly int $numero, ?Carbon $fecha = null)
    {
        $this->fecha = $fecha ?? Carbon::now();
    }

    public function haceCuanto(): string
    {
        return $this->fecha->locale("es")->diffForHumans();             // "hace 3 días"
    }

    public function fechaEntrega(): string
    {
        $entrega = $this->fecha->copy()->addWeekdays(3);                // 3 días laborables
        return $entrega->locale("es")->isoFormat("dddd D [de] MMMM");   // "lunes 9 de
noviembre"
    }
}
```

Calcular «hace 3 días» o «dentro de tres días laborables» con `DateTime` sería un buen rato de trabajo. Con Carbon, una línea, y en español.



🎯 Antes de instalar una librería, mirad en Packagist que esté **mantenida** (actualizada hace poco) y que la use mucha gente. Cada librería que instaláis es código ajeno dentro de vuestro proyecto.



CLASE 20 · PUNTO 8

⚠️ composer.json, composer.lock y vendor

Después de `composer require`, en vuestro proyecto hay tres cosas nuevas. Conviene saber qué es cada una:

Archivo o carpeta	Qué es	¿Se sube a Git?
`composer.json`	Lo que **vosotros** pedís: «Carbon, versión 3 o compatible»	✅ Sí
`composer.lock`	Lo que se ha instalado **exactamente**: «Carbon 3.15.1» y sus dependencias	✅ Sí
`vendor/`	El código descargado. Pesa mucho y se puede regenerar	❌ No: va en `.gitignore`

Y los comandos que vais a usar:

Comando	Qué hace	Cuándo
`composer require paquete`	Instala un paquete nuevo y lo apunta	Para añadir una librería
`composer install`	Instala **exactamente** lo que dice `composer.lock`	Al descargar un proyecto (por ejemplo, este repositorio)
`composer update`	Busca versiones nuevas y actualiza el `composer.lock`	Cuando queráis actualizar, a propósito
`composer dump-autoload`	Regenera la autocarga	Al cambiar la sección `autoload`
`composer run nombre`	Ejecuta un script del `composer.json`	⭐ Atajos propios (el ejemplo trae `composer run servidor`)



⚠️ **Al descargar los ejemplos de este repositorio**, la carpeta `vendor/` no viene (no se sube a Git). Entrad en la carpeta del proyecto con la terminal y ejecutad `composer install` antes de abrirlo en el navegador. Si no, veréis *Failed opening required 'vendor/autoload.php'*.



CLASE 20 · PUNTO 9

🐞 Los errores típicos de hoy

Mensaje	Qué ha pasado	Solución
`Failed opening required '.../vendor/autoload.php'`	No existe `vendor/`	`composer install` en la carpeta del proyecto
`Class "App\Modelos\Producto" not found`	El namespace y la carpeta no coinciden, o hay una errata	Revisad la regla PSR-4: namespace, carpeta y archivo, con las mismas mayúsculas
`Class "App\Modelos\DateTime" not found`	Una clase de PHP dentro de un archivo con namespace	`use DateTime;` o `new \DateTime()`
`Class "Producto" not found` (en `index.php`)	Falta el `use`	`use App\Modelos\Producto;` arriba del todo
`Cannot declare class Usuario, because the name is already in use`	Dos clases con el mismo nombre en el mismo namespace	Namespaces distintos, o un alias con `as`
`Namespace declaration statement has to be the very first statement`	Hay algo antes de `namespace`	Solo `<?php` y `declare(strict_types=1);` pueden ir antes
`composer: command not found`	La terminal no encuentra Composer	Usad la **Terminal de Laragon**, que ya lo tiene configurado



CLASE 20 · PUNTO 10

🚀 Ejercicios

Las soluciones 01 a 03 están en un mismo proyecto, `soluciones/academia/`. Recordad: `composer install` antes de probarlo.

- ☐ **01 · La academia con PSR-4.** Convertid las clases `Usuario`, `Alumno` y `Profesor` de la clase 18 en un proyecto Composer con el namespace `App\Usuarios` y la carpeta `src/Usuarios/`. El `index.php` no debe tener ni un `require` de clases. → `src/Usuarios/` e `index.php`
- ☐ **02 · Notificaciones.** Añadid un namespace `App\Notificaciones` con la interfaz `Notificador` y dos implementaciones (email y SMS) que reciban un `Usuario`. Avisad a todos los usuarios por los dos canales. → `src/Notificaciones/`
- ☐ **03 · Fechas con Carbon.** Instalad Carbon y mostrad la fecha de hoy en español, cuántos días (y cuántos laborables) quedan para el final de curso y, para cada alumno, cuánto hace que se dio de alta. → `fechas.php`

- ☐ **04 · Autocarga sin Composer.** Con las clases de la clase 18 **sin namespace**, escribid una función de autocarga que las busque en `clases/NombreDeLaClase.php`. Comprobad que el orden ya no importa. → `ej04-autocarga-sin-composer/`
- ☐ **05 · Una tienda desde cero.** Cread un proyecto nuevo con `composer init` (o a mano), con el namespace `Tienda\` en `src/`, y una clase `Tienda\Catalogo\Producto`. Generad la autocarga con `composer dump-autoload`. Las soluciones 05 a 10 están en el proyecto `soluciones/tienda/`. → `composer.json` y `Producto.php`
- ☐ **06 · Dos clases que se llaman igual.** `Tienda\Pedidos\Linea` y `Tienda\Facturas\Linea` en el mismo programa, sin choques, gracias a `use ... as`. → `Pedidos/Linea.php`, `Facturas/Linea.php` e `index.php`
- ☐ **07 · Funciones con namespace.** Una función `Tienda\euros()` en `src/funciones.php`. Las funciones no se autocargan con PSR-4: añadid el archivo a la sección `"files"` de `composer.json` y usadla con `use function`. → `funciones.php`
- ☐ **08 · Pagos en su namespace.** La interfaz `Tienda\Pagos\MetodoPago` y dos implementaciones, importadas de una vez con `use Tienda\Pagos\{Bizum, MetodoPago, Tarjeta};`. → `src/Pagos/`
- ☐ **09 · Un enum con namespace.** El enum `Tienda\Catalogo\Categoria` con una etiqueta para cada categoría, usado por `Producto`. → `Categoria.php`
- ☐ **10 · Un script de consola.** `bin/stock.php`, que carga la autocarga con `__DIR__ . "/../vendor/autoload.php"` y avisa de los productos agotados o con poco stock. Añadid un *script* en `composer.json` para lanzarlo con `composer stock`. → `bin/stock.php`



🏆 **Reto:** convertid el proyecto CRUD de la clase 16 en un proyecto Composer: un namespace `App\`, el archivo de consultas convertido en una clase `App\Repositorios\CursoRepositorio` y la autocarga con `vendor/autoload.php`. Es justo el camino que seguiremos en el MVC.



CLASE 20 · PUNTO 11

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
Namespace	El «apellido» de una clase: `App\Modelos\Usuario`
`use`	Importa una clase para llamarla por su nombre corto; `as` para un alias
Clases de PHP	En un archivo con namespace: `use DateTime;` o `\DateTime`
Autocarga	PHP llama a una función cuando no conoce una clase; esa función hace el `require`
PSR-4	Namespace = carpetas; clase = archivo. `App\` → `src/`
Composer	`composer.json` + `vendor/autoload.php` : un solo `require_once`
`composer require`	Instala una librería de Packagist
`composer install`	Instala lo que dice `composer.lock` (al descargar un proyecto)
Git	`composer.json` y `composer.lock` sí; `vendor/`, no

📚 Fuentes y documentación oficial

1. Manual de PHP: espacios de nombres
2. Manual de PHP: uso de espacios de nombres (importación y alias)
3. Manual de PHP: spl_autoload_register
4. PHP-FIG: PSR-4 Autoloader
5. Composer: Basic usage